



c2

Sorting vs. making

26 minutes

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Tell whether a problem needs a machine that sorts or a machine that makes — and say why.



The oldest split in the field

Almost everything in the taxonomy descends from these two questions.

Which bucket does this belong in?

Spam or not spam. Fraud or legitimate. Approve or decline.

MACHINES THAT SORT

What comes next?

Finish this sentence. Draft this email. Write this code.

MACHINES THAT MAKE

Two questions. Two entirely different kinds of machine, built differently and failing differently.



Machines that sort

Discriminative machine learning — the workhorse, and much older than anything you read about this year.

WHAT IT NEEDS

Labelled examples

Thousands of them. Someone had to say: this one is fraud, this one is not.

A fixed set of categories

Decided before you start training.

Data that looks like the future

It learns the world it was shown.

WHAT YOU GET

Narrow and very good

Inside its lane it beats everything, including a foundation model.

Cheap and fast

Milliseconds. Effectively free per prediction, once trained.

Fails predictably

You can tell when it is outside what it was trained on.

Every time your payment goes through in under a second, this is what checked it.



Machines that make

Generative AI — three years old in public, and already the fastest-adopted technology on record.

WHAT IT NEEDS

No labelled examples

None. You describe the task in a sentence.

Someone else's enormous investment

Roughly \$2M for a mid-size model; far more at the frontier.

A way to check the output

Because it will not tell you when it is wrong.

WHAT YOU GET

Broad and immediate

New task? Rewrite one sentence. No retraining, no data collection.

Costly per use

You pay per call, every call, forever.

Fails unpredictably — and confidently

The failure mode you met in C1.

53% of the world's population used one within three years — faster than the PC or the internet.

Source: Stanford HAI, AI Index Report 2026



[LIVE DEMO](#)

Same task. Both ways.

Sort customer reviews into positive and negative — once with a trained classifier, once with a prompt.

[screen share — Colab notebook]

Watch for the moment I change the number of training examples from 10 to 10,000.

Results on the next slide if the demo misbehaves.



What the demo showed

Same task, same data. The difference is not accuracy — it is what each one costs you.

	Trained classifier	Foundation-model API
Labelled examples needed	Thousands	None
Accuracy at 10 examples	Near useless	Already good
Accuracy at 10,000	Excellent	The same as before
Cost per 1,000 predictions	Effectively zero	Real money, every time
Speed	Milliseconds	About a second
Change the task	Retrain from scratch	Rewrite one sentence

Neither is better. They fail in opposite directions — one needs your data, the other needs your money.

So when would you use which?

USE A CLASSIFIER WHEN

- You have labelled data, or can get it
- The task will not change often
- Volume is high and margins are thin
- You need an answer in milliseconds
- You must explain the decision to a regulator

USE A FOUNDATION MODEL WHEN

- You have no labelled data
- The task changes constantly
- Volume is low, or you are still exploring
- The output is language, not a category
- You need it working this afternoon

And the honest answer for most real systems: both, in different places.



POLL · YOUR CALL

10 million support tickets. 5 categories.

You already have 50,000 correctly labelled examples. Which do you build?

- A** Trained classifier
- B** Foundation-model API
- C** Both — classifier first, model for the hard ones
- D** Not enough information to decide

Wooclap results

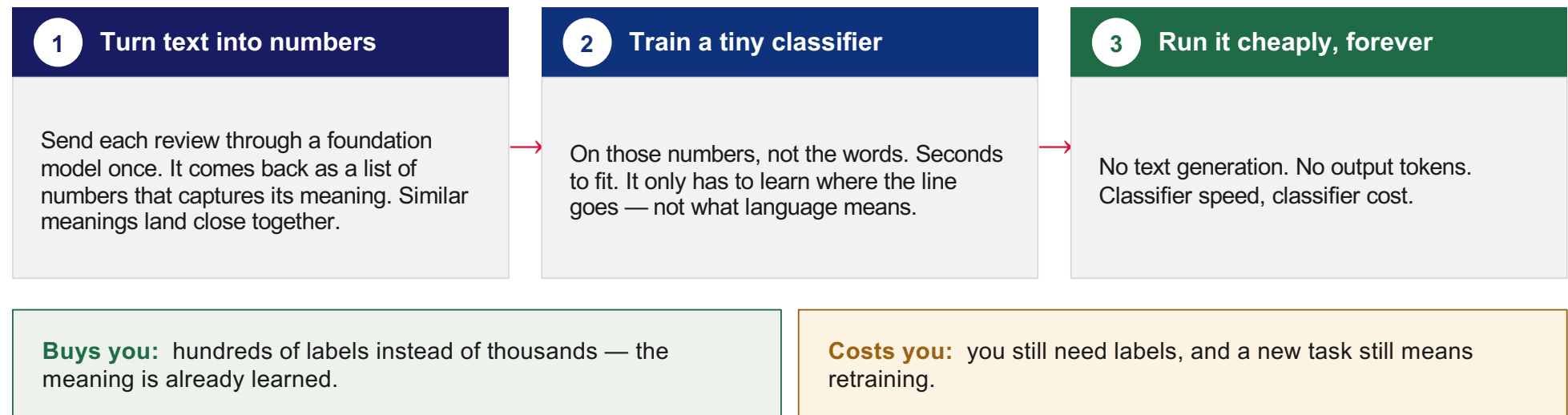
No correct answer. C and D are both defensible — the reasoning is the point.



What most real systems actually do

Those of you who picked C were right — it is not either/or. Here is the usual form of “both”.

The problem with the classifier was never the model. It was the words. “The delivery was fast” and “shipping arrived quickly” share almost no words — so it had to learn each phrasing separately.



Rent the understanding. Own the decision. · This is also how retrieval works — you build one in A1.

One of these needed thousands of your examples.
The other needed none at all.

How is that possible? Something changed in how these are built — and it is the reason the last three years look nothing like the twenty before them.

After the break: what changed.



Appendix — for the curious

Not part of the 26 minutes.

Why does a classifier beat a foundation model in its own lane?

It optimises a narrow decision boundary against your exact data distribution. A general model spends most of its capacity on things irrelevant to your task. Specialisation wins when the task is fixed and the data is representative.

Isn't the API just a classifier underneath?

No. It is doing next-token prediction, and you are choosing to read the output token as if it were a label. A classifier returns a calibrated probability for every class — 0.87 positive, 0.13 negative — so you can set a threshold and route anything below it to a human. The API returns the word “positive”, with no honest confidence attached. In production that difference decides whether you can automate at all.

What is actually inside an embedding vector?

A few hundred to a couple of thousand numbers. No individual number means anything you could name — there is no “politeness” dimension. What is meaningful is DISTANCE: the model was trained so that texts with similar meaning end up near each other. You are not reading the vector, you are measuring how close it is to others. That is also why you cannot inspect one to explain a decision.

